LLM-as-a-Judge

Automated Code Review at Scale

A practitioner's guide to replacing human code review with LLM-based evaluation, multi-judge aggregation, and VEAL loop integration for continuous validation.

6 Levels v1.0 Last updated June 2026

llm-as-a-judge code-review evaluation multi-agent VEAL automation

Human code review is the last bottleneck in autonomous software delivery. AI writes code faster than humans can read it. Teams using AI coding tools report 55% faster task completion—but the review queue grows longer, not shorter.

The Level 5 to Level 6 transition in the **Software Delivery Autonomy Model** requires removing this bottleneck. LLM-as-a-Judge is the pattern that makes it possible: structured evaluation frameworks where language models replace human reviewers, with calibration, bias mitigation, and deterministic escalation built in.

This article covers:

- **Why human review doesn't scale** with AI-generated code
- **Categorical scoring** and why it outperforms numeric scales
- **Multi-judge aggregation** for bias reduction
- **VEAL loop integration** for automated fix cycles
- **CI/CD patterns** for production deployment
- **Cost-benefit analysis** for enterprise adoption

The Code Review Bottleneck

Traditional code review has a fundamental scaling problem. As AI coding tools increase code velocity, review load increases proportionally. Teams report:

- Pull request queues growing despite faster code generation
- Senior engineers spending more time reviewing than building
- Review quality degrading as volume increases
- Merge-to-deploy cycles extending, not shrinking

The problem isn't that human review is bad—it's that human review is serial. One engineer reviews at a time. The throughput ceiling is low.

Anthropic's research on autonomous coding identified the critical insight:

> "Close the loop, give the model a way to verify its own output... The verification layer is equally important as generation capabilities—potentially more valuable for reliable deployment."

LLM-as-a-Judge closes that loop. Instead of humans reading every change, language models evaluate code against structured rubrics. Humans shift upstream to defining criteria, calibrating judges, and handling escalations.

Categorical Scoring: Pass, Partial, Fail

The first design decision in LLM-based evaluation is the scoring scale. Research and production experience converge on the same answer: **categorical scales outperform numeric scales** for LLM judges.

Why Numeric Scales Fail

Numeric scales (1-10, 1-5) create problems for LLM evaluation:

Calibration drift: What's a "7" versus an "8" varies by context and model version
Central tendency bias: Models cluster around middle values
Decision ambiguity: Does a "6" pass review or fail it?

The Three-Level System

Categorical scoring reduces these problems:

Score	Meaning	Decision Impact
Pass	Criteria fully met	Proceed without changes
Partial	Most requirements met, minor issues	Fix identified issues, then proceed
Fail	Missing key requirements or major issues	Block until resolved

This maps directly to workflow decisions. There's no ambiguity about whether code can merge.

Evidence-Based Reasoning

Each category result includes structured evidence:

```
CategoryResult{
    Category:  "error_handling",
    Score:     ScorePartial,
    Reasoning: "Most error paths handled, but missing context propagatio
    Evidence: [
        "Line 42: Error logged but not returned",
        "Line 89: Context not passed to downstream call"
    ],
    Findings: []Finding{
        {
            Severity:       SeverityMedium,
            Title:          "Missing error context",
            Location:       "pkg/api/handler.go:42",
            Recommendation: "Add error wrapping with context"
        }
    }
}
```

The reasoning chain and evidence make judge decisions auditable. When escalation happens, humans can trace exactly why.

Finding Severity: Blocking vs. Non-Blocking

Not all issues are equal. The severity system determines which findings block progress:

Severity	Blocking	Use Case
Critical	Yes	Security vulnerabilities, data loss risks
High	Yes	Functional bugs, API breaking changes
Medium	No	Code quality issues, missing tests
Low	No	Style inconsistencies, minor improvements
Info	No	Observations, no action required

This follows InfoSec conventions. Critical and High findings prevent merge. Medium and below are tracked but don't block.

Pass Criteria Configuration

Organizations configure their own pass criteria:

```
// Default: Zero critical or high findings
DefaultPassCriteria()  // 0 critical, 0 high, unlimited medium

// Strict: All categories pass, max 3 medium findings
StrictPassCriteria()   // All pass + max 3 medium

// Custom: Organization-specific thresholds
CustomPassCriteria{
    MaxCritical: 0,
    MaxHigh:     0,
    MaxMedium:   5,
    MinCategoriesPassing: "all_required"
}
```

Multi-Judge Aggregation

Single-judge evaluation inherits all the biases and failure modes of that model. Multi-judge aggregation reduces individual bias by running multiple evaluators on the same code.

Aggregation Methods

Method	Logic	When to Use
Majority	Most common score wins	General code review
Conservative	Lowest (most critical) score wins	Security-sensitive code
Optimistic	Highest score wins	Exploratory/experimental code
Unanimous	All judges must agree	High-stakes decisions

Agreement Metrics

Multi-judge evaluation produces agreement metrics that indicate reliability:

```
MultiJudgeResult{
    Evaluations:          [rubric1, rubric2, rubric3],
    AggregationMethod:    AggregationMajority,
    Agreement:            0.85,  // 85% agreement
    Disagreements: []JudgeDisagreement{
        {
            Category:     "performance",
            Scores:       []ScoreValue{Pass, Partial, Pass},
            FinalScore:   Pass,
        }
    },
    ConsolidatedDecision: DecisionPass,
}
```

When agreement drops below threshold (typically 0.6), the system flags for human review. Disagreement isn't failure—it's signal that the case needs human judgment.

Judge Configuration

Different models serve different purposes:

```
judges := []JudgeConfig{
    {Model: "claude-sonnet", Focus: "functional_correctness"},
    {Model: "claude-opus", Focus: "security_review"},
    {Model: "gpt-4", Focus: "api_design"},
}
```

Running diverse models catches model-specific blind spots.

Inter-Rater Reliability: Calibrating Judges

Before deploying LLM judges in production, calibrate them against human ground truth using Inter-Rater Reliability (IRR) metrics.

Key Metrics

Metric	Excellent	Good	Acceptable	Poor
Exact Agreement	>80%	60-80%	40-60%	<40%
Adjacent Agreement	>90%	70-90%	—	<70%
Pearson Correlation	>0.8	0.5-0.8	0.3-0.5	<0.3

Calibration Workflow

1. **Collect human judgments**: Sample of pull requests with human review scores
2. **Run LLM evaluation**: Same pull requests through candidate judges
3. **Compute IRR**: Compare LLM scores to human ground truth
4. **Tune prompts**: Adjust category definitions and examples until IRR meets threshold
5. **Monitor drift**: Continuously track agreement in production

If IRR falls below acceptable thresholds, the rubric or prompt needs adjustment—not deployment.

VEAL Loop Integration

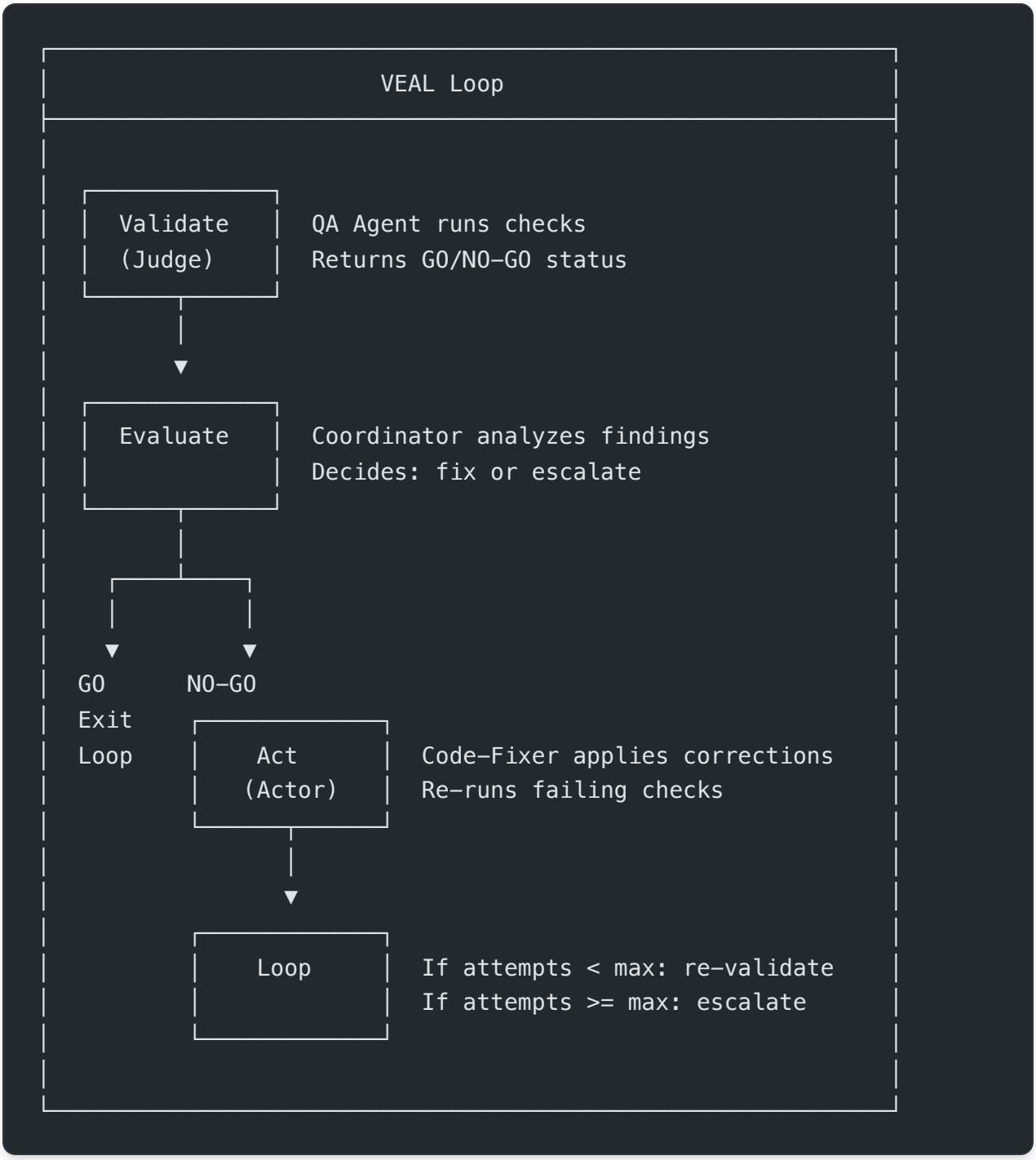
LLM-as-a-Judge reaches full power when integrated with **VEAL loops**. VEAL (Validate-Evaluate-Act-Loop) creates closed-loop systems where validation triggers automatic remediation.

The QA Fix Loop

The canonical VEAL implementation for code review:

```yaml
name: qa-fix
type: VEAL
description: Automated QA validation and fix cycle
validator: qa-agent          # LLM-as-Judge (read-only)
actor: code-fixer           # LLM-as-Actor (makes fixes)
max_attempts: 3
escalation: human
checks:
  - id: build
    type: command
    command: go build ./...
    required: true
  - id: lint
    type: command
    command: golangci-lint run
    required: true
  - id: tests
    type: command
    command: go test ./...
    required: true
  - id: error-handling
    type: pattern
    pattern: "_ = err"
    required: true
```

Execution Flow



Key Design Principles

Validator is read-only: The judge never modifies code. Separation prevents the judge from gaming its own evaluation.

Deterministic checks first: Run `go build`, `go test`, `golangci-lint` before LLM judgment. Deterministic failures don't need LLM interpretation.

Bounded attempts: Max 3 attempts prevents infinite loops. If the system can't converge, escalate to human.

Structured findings: Every failure produces actionable findings with location, severity, and recommendation.

Documentation Fix Loop

The same pattern applies to documentation validation:

```yaml
name: docs-fix
type: VEAL
validator: docs-reviewer     # Validates completeness
actor: docs-writer          # Creates/updates docs
max_attempts: 3
checks:
  - id: changelog
    type: file
    pattern: CHANGELOG.json
    required: true
  - id: readme-sections
    type: content
    check: README contains installation, usage, contribution
    required: true
  - id: release-notes
    type: file
    pattern: docs/releases/v*.md
    required: true
```

CI/CD Integration Patterns

LLM-as-a-Judge integrates into existing CI/CD pipelines as a validation stage.

GitHub Actions Example

```yaml
name: AI Code Review
on: [pull_request]

jobs:
  llm-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run LLM Review
        run: |
          sevaluation review \
            --rubric=code-quality \
            --judges=claude-sonnet,gpt-4 \
            --aggregation=conservative \
            --output=review-report.json

      - name: Check Pass Criteria
        run: sevaluation check review-report.json --strict

      - name: Post Review Comment
        if: always()
        run: |
          sevaluation render review-report.json --format=markdown \
            | gh pr comment ${{ github.event.pull_request.number }} --bod
```

Gate Configuration

Stage	Aggregation	Pass Criteria	Escalation
PR Review	Majority	0 critical, 0 high	Request changes
Pre-merge	Conservative	All categories pass	Block merge
Security	Unanimous	Zero security findings	Security team

Caching and Cost Control

LLM evaluation costs scale with code volume. Cost control strategies:

Incremental review: Only evaluate changed files, not full codebase
Tiered models: Use Haiku for fast checks, Sonnet for detailed review, Opus for security
Result caching: Cache evaluation results by file hash
Batch processing: Group small PRs for batch evaluation

Cost-Benefit Analysis

The Human Review Baseline

Senior engineer cost for code review:

Time: 30-60 minutes per substantial PR
Cost: $75-150/hour fully loaded
Throughput: 4-8 PRs/day maximum

At 100 PRs/week, that's 12-25 engineer-days of review labor weekly.

LLM Review Economics

Per-PR cost with multi-judge evaluation:

Component	Cost	Notes
Input tokens (3 judges)	$0.30-0.90	~10K tokens/PR average
Output tokens	$0.15-0.45	~2K tokens/PR average
Infrastructure	~$0.05	Compute, storage
Total per PR	**$0.50-1.50**	

At 100 PRs/week: **$50-150/week** versus **$15,000-37,500/week** for equivalent human review.

Break-Even Analysis

LLM-as-a-Judge pays for itself when:

- PR volume exceeds 20-30/week
- Human reviewers are senior (high opportunity cost)
- Review quality needs to be consistent across team
- 24/7 review availability is valuable

Hidden Costs to Consider

- **Calibration effort**: Initial rubric development and IRR calibration
- **Escalation handling**: Humans still review flagged cases
- **Model upgrades**: Re-calibration when underlying models change
- **Edge case coverage**: Complex architectural decisions may need human judgment

---

Production Implementation

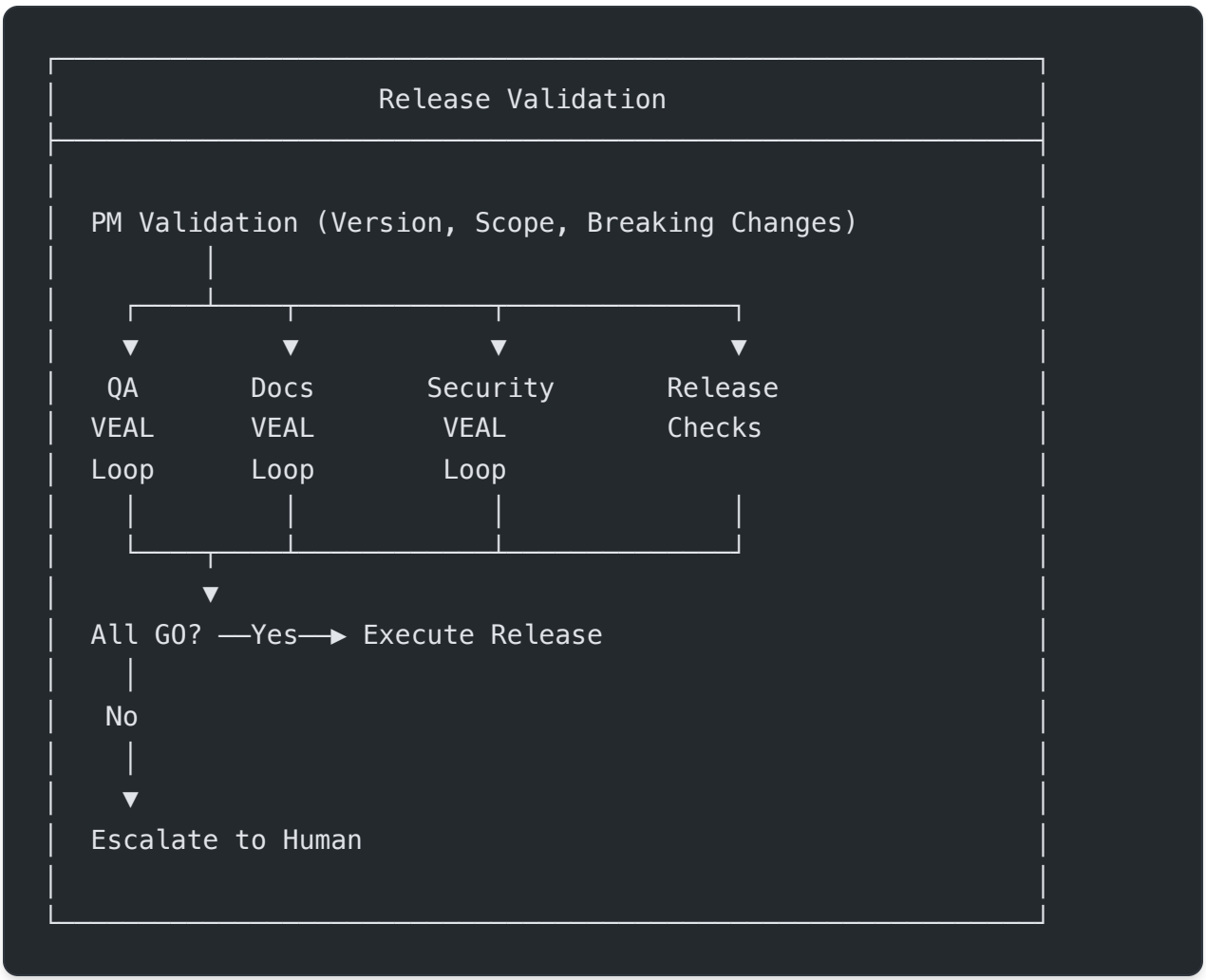
The Structured Evaluation Stack

A production LLM-as-a-Judge system includes:

Rubric definitions: Category-specific evaluation criteria with examples
Judge metadata: Model, temperature, prompt version tracking for reproducibility
Multi-judge aggregation: Configurable aggregation with disagreement detection
Report generation: Terminal, markdown, JSON output formats
Loop integration: VEAL loop support for automated remediation

Example: Release Validation

A complete release validation pipeline using LLM-as-a-Judge:



Each validation area runs its own VEAL loop with judge and fixer agents. The coordinator orchestrates all loops and only proceeds when all areas report GO.

The Level 6 Enabler

LLM-as-a-Judge is the technical foundation for ASDM Level 6: Autonomous Coding & Review. Without automated evaluation, human review remains the bottleneck regardless of how fast AI generates code.

The shift is organizational as much as technical:

Aspect	Level 5	Level 6
Quality gate	Human review	Scenario satisfaction
Human role	Reviewer	Criteria designer
Throughput	Limited by reviewers	Limited by compute
Consistency	Varies by reviewer	Deterministic

Organizations at Level 5 have AI writing code with human review. Organizations at Level 6 have AI writing and reviewing code, with humans designing the criteria and handling escalations.

The transition requires investment in:

Rubric engineering: Translating review standards into machine-evaluable criteria
Calibration infrastructure: IRR measurement and continuous monitoring
Escalation workflows: Clear handoff when automation can't decide
Cultural change: Trust in automated evaluation when properly calibrated

---

Conclusion

Human code review was designed for human-written code at human pace. AI-generated code at AI pace breaks that model.

LLM-as-a-Judge provides the alternative:

Categorical scoring for unambiguous decisions
Multi-judge aggregation for bias reduction
IRR calibration for production reliability
VEAL loop integration for automated remediation
CI/CD patterns for seamless deployment

The goal isn't to remove humans from software delivery. It's to move humans to where they add the most value: defining what good looks like, not manually verifying every change.

For teams operating at high code velocity with AI generation, LLM-as-a-Judge isn't optional—it's the path from bottleneck to scale.